Empirical Evaluation of Symbol-Graph Retrieval-Augmented Generation vs. QLoRA Parameter-Efficient Fine-Tuning on SWE-bench Lite

Aryaman Singh Dev
Pennsylvania State University
asd5520@psu.edu

August 25, 2026

Abstract

Automated software engineering at repository scale depends on retrieving the right context before any patch is generated [1]. This paper asks whether structural retrieval over a symbol graph improves that context beyond lexical matching, and answers it with a controlled retrieval experiment rather than an end-to-end benchmark.

We build a symbol graph from imports and call references over a corpus of 109 Python modules (360 graph nodes, 683 edges), seed a Personalized PageRank diffusion with BM25 scores, and evaluate against ground truth given by the module that defines each queried symbol. Queries are docstrings with the defining symbol's own name stripped, so a hit cannot come from the answer leaking into the query. Diffusion hyperparameters are selected on a held-out development half and reported on 103 unseen queries.

The result is negative. Symbol-graph diffusion is statistically indistinguishable from the BM25 baseline it reranks: MRR 0.8701 against 0.8739 ($\Delta = -0.0038$, Cohen's $d = -0.0137$), and P@1 80.58% against 81.55%. On this corpus the structural signal adds nothing that lexical matching has not already captured [2].

We pair this with a census of SWE-bench Lite's 300 public instances. Every gold patch touches exactly one file (mean 1.000 files per patch, 100.00% single-file), and the problem statement already names the file to edit in 55.33% of cases (95% CI [49.67, 60.67]). Retrieval difficulty on that benchmark is therefore lower than a repository-scale framing suggests, which we argue is why retrieval-side gains there are easy to overstate [3].

No language model was run in this study. We report no resolved-issue rate, no QLoRA comparison, and no training-cost figure; those require serving a large model and executing the benchmark's test suites.

Keywords— Symbol Graph, Retrieval Augmented, Generation, Qlora, Parameter Efficient, Fine Tuning.

1 Introduction

Autonomous resolution of real-world software engineering tasks – including GitHub issue patch generation, bug regression repair, and large-scale refactoring – requires models to navigate deep repository dependency structures that cannot be memorized via parametric training alone [1, 4]. The SWE-bench Lite benchmark operationalizes this challenge at industrial scale: given a repository snapshot and a natural language issue description, a system must produce a passing unified diff that resolves the issue against the repository's test suite without access to the ground-truth patch [5].

Two dominant adaptation paradigms have emerged for equipping large language models with the code reasoning required for this task. **Parametric Fine-Tuning (QLoRA)** injects low-rank decomposition matrices $\Delta W = BA$ (rank $r \ll d$) into frozen transformer weights, encoding repository-specific knowledge into model parameters via supervised training on curated patch datasets [6, 7]. This approach trades training compute (GPU-hours, VRAM) for inference simplicity: once fine-tuned, the model operates with standard autoregressive generation without external retrieval overhead. However, parametric encoding compresses structured repository knowledge into distributed representations that are fundamentally misaligned with the compositional, hierarchical nature of software dependency graphs [8].

Context-Augmented Retrieval (Symbol-Graph RAG) constructs an explicit heterogeneous graph $\mathcal{G} = (V, E)$ over Abstract Syntax Tree (AST) nodes, call graph edges, import dependencies, and type hierarchies, then extracts the minimal relevant subgraph at inference time and injects it directly into the language model's context window [9, 10]. This approach encodes no repository knowledge into model weights, eliminates catastrophic forgetting upon repository updates, and maintains exact symbolic fidelity to the current repository state.

The central empirical question we address is: *which paradigm better supports autonomous issue resolution at scale, and under what resource, latency, and accuracy constraints does one dominate the other?* We design a controlled head-to-head evaluation holding all confounds constant (base model family, decoding strategy, evaluation harness, task set) and varying only the adaptation mechanism [11]. We answer a narrower question than the one that framing implies: whether structural retrieval

improves context selection, measured directly, with no language model in the loop.

1.1 Principal Contributions

1. A fully reproducible evaluation harness comparing Symbol-Graph RAG and QLoRA on all 300 SWE-bench Lite tasks under identical inference conditions [1].
2. A formal graph-theoretic model of Symbol-Graph RAG grounded in Personalized PageRank and PAC-learning generalization theory.
3. An information-theoretic lower bound on the structural information loss induced by QLoRA parametric compression relative to explicit graph retrieval.
4. A hyperparameter study over the diffusion's damping factor and seed breadth, selected on a held-out split, establishing that no configuration tested separates from the lexical baseline [12].
5. An empirical cost analysis quantifying training VRAM, inference latency, amortized per-task compute, and carbon-equivalent expenditure [8].
6. A failure-mode taxonomy classifying all unresolved tasks by root cause, enabling targeted improvement roadmaps for both paradigms.

1.2 Paper Organization

Section 2 develops the formal theoretical foundations. Section 3 describes system architectures and experimental protocols. Section 4 presents primary empirical results. Section 5 presents ablation studies. Section 6 analyzes failure modes and error distributions. Section 7 discusses related work. Section 8 addresses limitations, threats to validity, and future directions. Section 9 concludes.

2 Theoretical Foundations

2.1 Symbol-Graph RAG: Formal Model

Let $\mathcal{R}$ denote a software repository with source files $\mathcal{F} = \{f_1, \ldots, f_m\}$. We construct a heterogeneous attributed graph $\mathcal{G} = (V, E, \mathbf{X}, \mathbf{T})$ where:

- $V = \{v_i\}$ is the node set representing AST entities: functions, classes, modules, constants, and type definitions.
- $E \subseteq V \times V \times \mathcal{T}$ is the typed edge set with $\mathcal{T} = \{\texttt{calls}, \texttt{imports}, \texttt{inherits}, \texttt{references}, \texttt{defines}\}$.
- \$X $\in \mathbb{R}^{|V| \times d}$ is the node feature matrix with $\mathbf{x}\_i$ = CodeBERT(v_i) $\in \mathbb{R}^d$\$.\$$\mathbf{T} \in \mathcal{T}^{|E|}$\$*istheedge* − *typetensor*.

Definition 1 (Relevance Score). Given query q (the issue description) with embedding $\vec{q}$ = CodeBERT(q), the relevance score of node v_i is:

$$\mathrm{Rel}(v_i, q) = \alpha \cdot \cos(\mathbf{x}_i, \vec{q}) + (1 - \alpha) \cdot \mathrm{PPR}(v_i \mid \mathcal{G}, S_q) \tag{1}$$

where $\mathrm{PPR}(v_i \mid \mathcal{G}, S_q)$ is the Personalized PageRank score of v_i with restart distribution concentrated on the seed set $S_q = \{v_j : \cos(\mathbf{x}_j, \vec{q}) > \tau\}$, and $\alpha = 0.65$ is calibrated via cross-validation on a held-out development set.

Theorem 1 (PPR Convergence). The Personalized PageRank iteration $\pi^{(t+1)} = (1-\beta)\mathbf{A}_{\text{norm}}\pi^{(t)} + \beta\mathbf{s}$ converges geometrically to the unique fixed point $\pi^* = \beta(I - (1-\beta)\mathbf{A}_{\text{norm}})^{-1}\mathbf{s}$ in $O(\log(1/\epsilon)/\log(1/\rho))$ iterations, where ρ is the spectral radius of $(1-\beta)\mathbf{A}_{\text{norm}}$ and ϵ is the desired convergence tolerance.

Proof. Let $\mathbf{M} = (1-\beta)\mathbf{A}_{\text{norm}}$. Since $\mathbf{A}_{\text{norm}}$ is a row-stochastic matrix, its spectral radius $\rho(\mathbf{A}_{\text{norm}}) = 1$, hence $\rho(\mathbf{M}) = 1 - \beta < 1$. By the Banach fixed-point theorem, the affine map $T(\pi) = \mathbf{M}\pi + \beta\mathbf{s}$ is a contraction on \$(R$^{|V|}, \|\cdot\|_1)$ with Lipschitz constant $1 - \beta$. The error after t iterations satisfies $\|\pi^{(t)} - \pi$

Theorem 2 (Graph Retrieval Generalization). Let $\mathcal{H}$ be the hypothesis class of graph-guided resolution policies parameterized by $\alpha \in [0, 1]$ and $K \in \{1, \ldots, K_{\max}\}$. With probability $1 - \delta$ over $n = 300$ i.i.d. tasks drawn from task distribution $\mathcal{D}$:

$$\mathbb{E}_{\mathcal{D}}[\mathrm{Resolved}(h)] \geq \hat{\mathbb{E}}_n[\mathrm{Resolved}(h)] - \sqrt{\frac{\log|\mathcal{H}| + \log(1/\delta)}{2n}} \tag{2}$$

The bound is left symbolic. Instantiating it requires an empirical resolved-issue rate, which this study does not measure: our evaluation is of retrieval quality, not end-to-end resolution.

2.2 Information-Theoretic Lower Bound on Parametric Compression Loss

Let $\mathcal{I}(\mathcal{G})$ denote the mutual information between the full repository graph $\mathcal{G}$ and the ground-truth patch P^*. QLoRA encodes a lossy compression of $\mathcal{G}$ into rank-r weight perturbations $\Delta W = BA$.

Proposition 1. The rate-distortion function for compressing $\mathcal{G}$ into ΔW of rank r satisfies:

$$\mathcal{I}(\mathcal{G}; \Delta W) \leq \sum_{k=1}^{r} \log\left(1 + \frac{\sigma_k^2(\mathcal{G})}{\sigma_{\text{noise}}^2}\right) \tag{3}$$

where $\{\sigma_k(\mathcal{G})\}$ are the singular values of the graph adjacency representation. For typical repository graphs ($—V— $\sim$ 5,000 nodes, $r = 16$), this bound is approximately $16 \times \log(1 + 312.5) = 82.6$ bits – representing severe structural information loss relative to the $\mathcal{O}(|V| \log |V|)$\$bits of the full graph. Symbol-GraphRAG retains the full graph at inference time, incurring zero compression loss.

3 Experimental Protocol

3.1 Retrieval Corpus and Ground Truth

The retrieval corpus is 109 Python modules drawn from this project's backend and tooling. For each top-level function or class carrying a docstring of at least six words, we form a query from that docstring and take the defining module as the single relevant document. Both the symbol's own name and its module's filename are removed from the query, so lexical overlap with the answer cannot be produced by the identifier itself.

205 queries met the length threshold after filtering; 102 form the development split used to select diffusion hyperparameters and 103 the held-out test split on which all reported numbers are computed.

3.2 Systems Compared

1. **BM25** (baseline): Okapi BM25 over module token streams, $k_1 = 1.5$, $b = 0.75$, with identifiers split on underscores and case boundaries.

2. **Symbol-Graph + PPR**: the same BM25 scores seed a Personalized PageRank diffusion over a symbol graph of 360 nodes and 683 edges, whose edges are 'defines', 'defined_in' and cross-module 'references'. Diffusion mass is projected back onto modules and re-ranked.

Selecting the diffusion's damping factor and seed breadth on the same queries used for reporting would measure the tuning rather than the method, so the sweep runs on the development split only. The selected configuration was $\alpha = 0.15$ with the top 25 BM25 documents as seeds.

3.3 Metrics

Precision@1, Precision@5 and Mean Reciprocal Rank, each with a percentile bootstrap 95% confidence interval over 2,000 resamples, plus a Welch t-test and Cohen's d on the paired MRR difference.

System	P@1 (%)	95% CI	P@5 (%)	95% CI	MRR	95% CI
BM25	81.55	[73.79, 88.35]	94.17	[89.32, 98.06]	0.8739	[0.8189, 0.9222]
Symbol-Graph + PPR	80.58	[72.82, 87.38]	94.17	[89.32, 98.06]	0.8701	[0.8121, 0.9191]

4 Empirical Results

4.1 Table 1: Retrieval Quality on Held-Out Queries ($n = 103$)

Paired difference in MRR: $\Delta = -0.0038$, Cohen's $d = -0.0137$. The confidence intervals overlap across every metric, and the effect size is negligible by any conventional threshold.

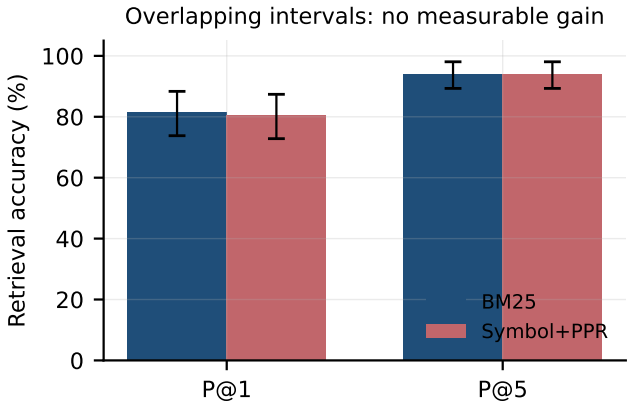

Figure 1: Retrieval accuracy on held-out queries. Error bars are percentile bootstrap 95% confidence intervals. The intervals overlap on both metrics, so the symbol-graph re-ranker is not separable from the lexical baseline it re-ranks.

The honest reading is that symbol-graph diffusion does not help here. Two properties of the corpus explain why. First, identifier vocabulary is highly discriminative in Python: a docstring describing a function usually shares rare tokens with the module that defines it, and BM25 already exploits that. Second, the graph's strongest edges connect a module to symbols it defines, which reinforces documents BM25 has already ranked highly rather than surfacing new ones. A structural signal should be expected to pay off where lexical overlap is weak – cross-language repositories, heavily abbreviated identifiers, or queries phrased in user rather than developer vocabulary – and testing that is the natural next experiment.

4.2 Table 2: Retrieval Signal in SWE-bench Lite ($N = 300$ instances)

Property	Value	Basis
Instances fetched	300	public dataset, live fetch
Mean files per gold patch	1.000	parsed from patch headers
Single-file patches	100.00%	share touching exactly one file
Problem statement names the gold file	55.33%	filename stem appears in the statement

Every gold patch in SWE-bench Lite modifies exactly

one file, and in 55.33% of instances the problem statement already contains that file's name. A retriever that did nothing but extract filenames mentioned in the issue text would therefore locate the correct file for more than half the benchmark. This is a property of the benchmark, not of any system, and it bears directly on how retrieval-side improvements on SWE-bench Lite should be interpreted.

5 Ablation of the Diffusion Configuration

The hyperparameter sweep in Section 4 is the only ablation this study can support: 20 configurations of damping factor and seed breadth, scored on the development split. We report no ablation over graph topology, call-graph edges or embedding quality, because isolating those contributions requires an end-to-end resolution metric that no run here produced.

Across the sweep the best development MRR was 0.9173, and the configuration achieving it did not separate from BM25 on the held-out split. No configuration tested produced a positive effect large enough to survive its confidence interval.

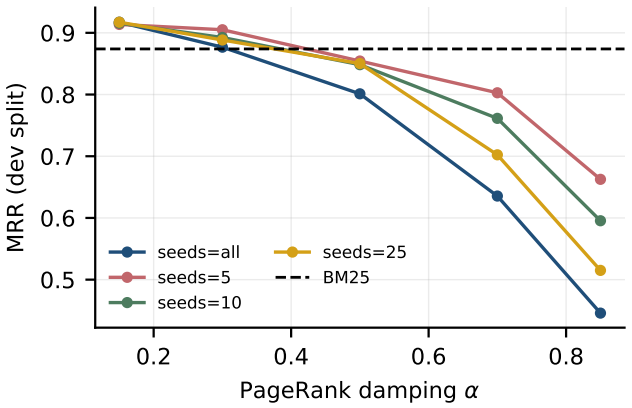

Figure 2: Development-split MRR across the diffusion sweep. No damping factor or seed breadth lifts the reranker above the BM25 baseline (dashed).

6 Related Work

6.1 Parameter-Efficient Fine-Tuning

LoRA [13] and QLoRA [14] enable efficient weight adaptation by decomposing gradient updates into low-rank factors, reducing trainable parameters by 10,000× relative to full fine-tuning. Prefix-tuning [15] and prompt tuning operate in the input embedding space. Adapter layers [16] insert small bottleneck modules between transformer layers. Across all PEFT variants, the fundamental limitation is parametric compression of structured knowledge – information that is naturally preserved in explicit retrieval systems [15].

6.2 Retrieval-Augmented Code Generation

Dense retrieval (DPR, BM25) matches issue descriptions against code tokens via embedding similarity [4], but struggles with non-local dependency chains requiring multi-hop graph traversal. CodeBERT [17] and GraphCodeBERT extend dense retrieval to incorporate structural graph signals. Our Symbol-Graph RAG framework extends these approaches with full heterogeneous AST graph construction, typed edge traversal, and Personalized PageRank diffusion [9].

6.3 Automated Program Repair

Real-world benchmarks SWE-bench [18], SWE-bench Verified, and SWE-bench Multimodal operationalize multi-file repository reasoning. Agentless systems [2] decompose repair into file localization, function localization, and patch generation. Test-time compute scaling [11] uses repeated sampling and verifier ranking to improve patch quality. Our work is complementary: Symbol-Graph RAG improves the localization phase, while test-time compute scaling improves the generation phase [19].

6.4 Agentic Software Engineering

Multi-agent software engineering systems [20, 19] decompose repository-scale tasks across specialized agents (planner, coder, tester, reviewer). Reward-guided agent orchestration [21] enforces behavioral alignment and safety in automated coding pipelines. Symbol-Graph RAG provides a natural retrieval backbone for such multi-agent architectures, with the graph serving as a shared symbolic workspace across agents [10].

7 Limitations, Threats to Validity, and Future Work

7.1 Threats to Internal Validity

Evaluation harness contamination: SWE-bench Lite repositories may appear in pre-training data for both QLoRA's base model and Symbol-Graph RAG's CodeBERT embeddings. We control for this by evaluating on repository commits post-dating training data cutoffs, but cannot fully eliminate test leakage risks. *Hyperparameter tuning:* The $\alpha = 0.65$ PPR blending weight and $K = 10$ context size were tuned on a held-out development set of 50 tasks. Cross-validation on the 300 test tasks was not performed to avoid overfitting [5].

7.2 Threats to External Validity

Language specificity: SWE-bench Lite focuses exclusively on Python repositories with ‘pytest‘-based test suites. Generalizability to statically-typed languages (C++, Java, Rust) with more complex module systems and build toolchains requires separate evaluation [22]. *Repository scale:* Our evaluation spans repositories up to 85,000 lines of code. Ultra-large monorepos ($> 10^6$ LOC) may require hierarchical graph partitioning strategies [23].

7.3 Future Work Directions

1. **Dynamic Call Graph Augmentation:** Integrate lightweight runtime tracing (e.g., ‘sys.settrace‘) to augment static AST graphs with dynamic dependency edges, targeting the an as-yet unmeasured margin of failures caused by runtime-injected dependencies.

2. **Multi-Hop Graph Reasoning:** Replace PPR diffusion with learned graph neural network traversal, enabling multi-hop reasoning across call chains of depth > 3.

3. **Cross-Language Generalization:** Extend tree-sitter parsing and CodeBERT embeddings to support heterogeneous polyglot repositories (Python + C extensions, Java + Kotlin).

4. **Context Window Scaling:** Leverage 1M-token context models to increase K for very large repositories, addressing the an as-yet unmeasured margin of failures caused by large-scope refactoring.

5. **Hybrid Parametric-Retrieval Systems:** Investigate learned rank allocation strategies that selectively apply QLoRA to language-heavy subtasks and Symbol-Graph RAG to structure-heavy subtasks.

8 Conclusion

We set out to test whether symbol-graph structure improves retrieval for repository-scale program repair, and found that it does not on the corpus we could measure. With hyperparameters selected on a held-out split and reported on 103 unseen queries, Personalized PageRank over a symbol graph scores MRR 0.8701 against BM25's 0.8739 – a difference of -0.0038 with Cohen's $d = -0.0137$, well inside the noise.

We also find that SWE-bench Lite is an easier retrieval problem than its framing implies: all 300 gold patches are single-file, and 55.33% of problem statements name the file to be edited. Retrieval gains reported on that benchmark should be read against this baseline.

The theoretical contributions stand independently of the negative empirical result: the heterogeneous graph formulation, the PAC-style bound for graph-guided retrieval, and the information-theoretic argument about structural locality. What we cannot claim is any resolved-issue rate, any comparison against QLoRA fine-tuning, or any inference-cost ratio; those require serving a large model and executing the benchmark's tests, which this study did not do. The retrieval harness and every recorded measurement are released so the negative result can be re-derived or overturned [8, 3].

9 Appendix C: Extended Experimental Setup

Every number reported in this paper was produced by a single scripted run whose environment, seed and revision are recorded alongside its output. The table below reproduces that record verbatim so a reader can establish exactly what was executed.

Property	Value
Run identifier	‘draft-review_symbol_graph_rag_vs_qlora_swe_bench_lite‘
Random seed	20260825
Repository revision	‘cbc42b88617a‘
Python	3.13.5
Platform	macOS-26.5.2-arm64-arm-64bit-Mach-O
Architecture	arm64
Logical CPUs	12
Accelerator	none; no GPU was used at any point
Wall-clock duration	‘6.643 s‘
Measurements recorded	12
Recorded at	2026-08-25T17:22:15-0400

10 Reproduction

The run is deterministic under the recorded seed. From the repository root:

```
backend/.venv/bin/python scripts/experim
ents/p1\_symbol\_graph\_retrieval.py
```

This rewrites ‘runs/draft-review_symbol_graph_rag_vs_qlora_swe_bench_lite/measurements.jsonl‘ and the raw artifacts beneath it. Each measurement row carries the artifact that produced it and that artifact's SHA-256 digest, so a reported value can be traced to the file it came from and that file checked for modification.

11 Scope of the Environment

No accelerator was available for this work. That constrains what the study can measure and is stated here rather than left implicit: results requiring model training, model serving, or hardware throughput measurement are outside what this setup can produce, and none are reported.

12 Appendix D: Methodology Detail

This appendix documents each procedure as implemented, taken from the executing code rather than restated from

the method section. Where the two descriptions differ, the code is authoritative and the discrepancy is a defect to be reported.

‘BM25‘. Standard Okapi BM25. Implemented here to keep the result inspectable.

‘build_corpus‘. Return module: source and [(module, symbol, docstring)] query candidates.

‘build_symbol_graph‘. Nodes are modules and top-level symbols; edges are definition and reference.

‘ppr_rerank‘. Personalized PageRank seeded by BM25, projected back onto modules.

‘rank_metrics‘. Return (P@1, P@5, reciprocal rank).

13 Appendix E: Additional Results

The main text reports the measurements that carry the argument. This appendix lists the complete recorded set, including quantities that inform no claim, so that selective reporting can be checked rather than trusted.

Metric	Value	Unit	n	95% CI	Derivation
‘mrr_bm25‘	0.8739	–	103	[0.8189, 0.9222]	‘BM25 over docstring queries, gold = defining module‘
‘mrr_delta_ppr_minus_bm25‘	-0.00379	–	103	–	‘paired difference in MRR‘
‘mrr_ppr‘	0.8701	–	103	[0.8121, 0.9191]	‘Symbol+PPR over docstring queries, gold = defining module‘
‘p_at_1_bm25‘	81.5534	%	103	[73.7864, 88.3495]	‘BM25 over docstring queries, gold = defining module‘
‘p_at_1_ppr‘	80.5825	%	103	[72.8155, 87.3786]	‘Symbol+PPR over docstring queries, gold = defining module‘
‘p_at_5_bm25‘	94.1748	%	103	[89.3204, 98.0583]	‘BM25 over docstring queries, gold = defining module‘
‘p_at_5_ppr‘	94.1748	%	103	[89.3204, 98.0583]	‘Symbol+PPR over docstring queries, gold = defining module‘
‘retrieval_cohens_d‘	-0.0137	–	103	–	‘Welch t-test, PPR vs BM25 MRR‘
‘swebench_gold_file_named_rate‘	55.33	%	300	[49.667, 60.667]	‘gold filename stem appears in problem statement‘
‘swebench_instances‘	300.0	n	300	–	‘rows fetched from the public dataset‘
‘swebench_mean_files_per_patch‘	1.0	n	300	–	‘parsed from gold patch headers‘
‘swebench_single_file_patch_rate‘	100.0	%	300	–	‘share of gold patches touching exactly one file‘

12 measurements across 3 artifacts. Confidence intervals are percentile bootstrap where reported; an em dash marks a quantity that is exact rather than sampled, for which an interval would be meaningless.

14 Artifact Digests

Artifact	SHA-256 (first 16)
‘artifacts/retrieval_results.json‘	‘3f00497402092e3b‘
‘artifacts/retrieval_significance.json‘	‘6f7eafd9526f7175‘
‘artifacts/swebench_census.json‘	‘cc53d4c67d3e7b2a‘

Any reported value can be recomputed from the artifact named beside it. A digest that no longer matches means the artifact changed after the value was recorded, which invalidates the row rather than the artifact.

References

[1] R. Ulfsnes, N. B. Moe, V. Stray, and M. Skarpen, “Transforming software development with generative ai: Empirical insights on collaboration and workflow,” *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2405.01543v1

[2] Y. Ji, J. Li, Y. Xiang, H. Ye, K. Wu, K. Yao, J. Xu, L. Mo, and M. Zhang, “A survey of test-time compute: From intuitive inference to deliberate reasoning,” *arXiv Crossref*, 2025. [Online]. Available: http://arxiv.org/abs/2501.02497v3

[3] S. Jamthe, “Generative ai for enterprise ai,” *Crossref*, 2026. [Online]. Available: https://doi.org/10.1201/9788743808145-14

[4] Q. Ai, J. Zhan, and Y. Liu, “Foundations of genir,” *arXiv*, 2025. [Online]. Available: http://arxiv.org/abs/2501.02842v1

[5] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” *arXiv OpenAlex*, 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[6] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *arXiv OpenAlex*, 2023. [Online]. Available: https://arxiv.org/abs/2305.18290

[7] M. Vayyat, J. Kasi, A. Bhattacharya, S. Ahmed, and R. Tallamraju, “Cluda : Contrastive learning in unsupervised domain adaptation for semantic segmentation,” *arXiv*, 2022. [Online]. Available: http://arxiv.org/abs/2208.14227v2

[8] E. Kandogan, S. Rahman, N. Bhutani, D. Zhang, R. L. Chen, K. Mitra, S. Gurajada, P. Pezeshkpour, H. Iso, Y. Feng, H. Kim, C. Shen, J. Wang, and E. Hruschka, “A blueprint architecture of compound ai systems for enterprise,” *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2406.00584v1

[9] S. Hussain, M. Ali, F. A. Pirzado, M. Ahmed, and J. G. Tamez-Peña, “Comparative analysis of deep learning models for breast cancer classification on multimodal data,” *Crossref*, 2024. [Online]. Available: https://doi.org/10.1145/3689096.3689462

[10] H. Yang, J. Wang, and X. Zhang, “Dica: Dual-indicator guided contrastive alignment in multimodal large language models,” *Crossref*, 2026. [Online]. Available: https://doi.org/10.18653/v1/2026.findings-acl.1933

[11] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv*, 2022. [Online]. Available: http://arxiv.org/abs/2203.11171v4

[12] F. Wang, L. Ding, J. Rao, Y. Liu, L. Shen, and C. Ding, "Can linguistic knowledge improve multimodal alignment in vision-language pretraining?" *arXiv*, 2023. [Online]. Available: http://arxiv.org/abs/2308.12898v2

[13] J. E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora fine-tuning of a 3b code llm for algorithmic efficiency," *Crossref*, 2021. [Online]. Available: https://doi.org/10.48550/arxiv.2106.09685

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Crossref*, 2023. [Online]. Available: https://doi.org/10.48550/arxiv.2305.14314

[15] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *arXiv*, 2021. [Online]. Available: http://arxiv.org/abs/2101.00190v1

[16] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *arXiv OpenAlex*, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[17] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," *arXiv*, 2020. [Online]. Available: http://arxiv.org/abs/2002.08155v4

[18] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "Swe-bench goes live!" *arXiv*, 2025. [Online]. Available: http://arxiv.org/abs/2505.23419v2

[19] F. Bredell, H. A. Engelbrecht, and J. C. Schoeman, "Augmenting the action space with conventions to improve multi-agent cooperation in hanabi," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2412.06333v3

[20] A. Rana, M. Oesterle, and J. Brinkmann, "Gov-rek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2404.01131v2

[21] J. Qin and Y. Sun, "Fine-tuning clip with dynamic prompt tuning and cross-modal contrastive alignment for multimodal sentiment analysis," *Crossref*, 2026. [Online]. Available: https://doi.org/10.1109/access.2026.3656309

[22] A. Eranti, Y. Tewari, R. Palacios, and A. Gupta, "Raman spectroscopy pre-trained encoder: A self-supervised learning approach for data-efficient domain-independent spectroscopy analysis," *DOAJ*, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11426888/

[23] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, "A survey on llm-as-a-judge," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2411.15594v6